

《漏洞利用及渗透测试基础》实验报告

姓名：谢小珂 学号：2310422 班级：1069

实验名称：

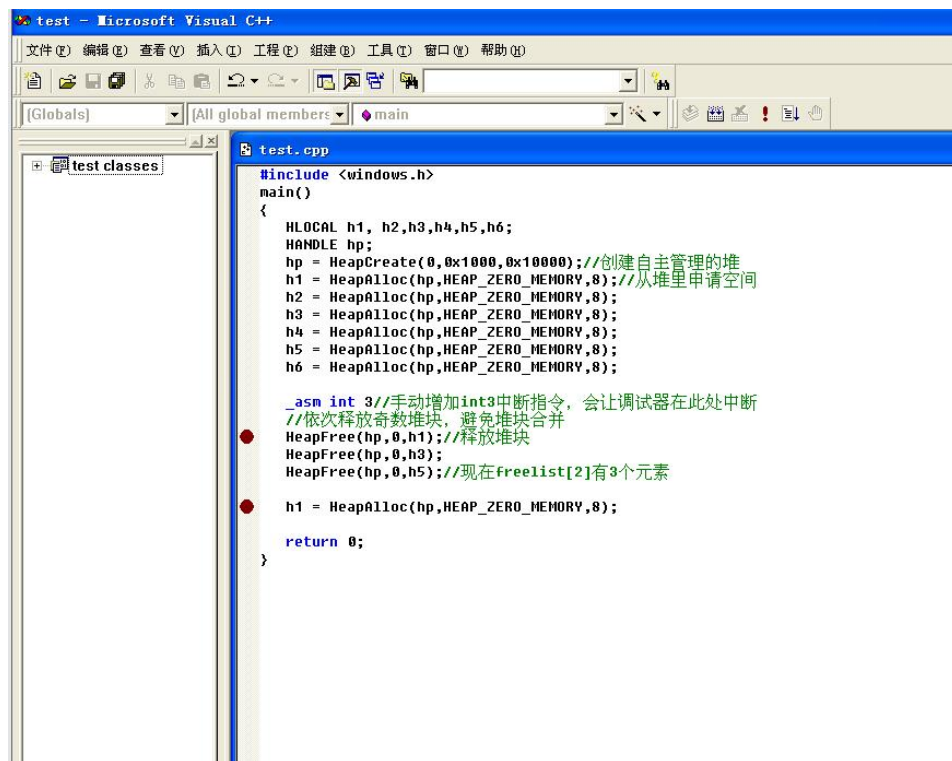
堆溢出 Dword Shoot 模拟实验

实验要求：

以第四章示例 4-4 代码为准在 VC IDE 中进行调试，观察堆管理结构，记录 Unlink 节点时的双向空闲链表的状态变化，了解堆溢出漏洞下的 Dword Shoot 攻击。

实验过程：

实验通过释放和重新分配堆块，构造了空闲链表 freelist[2]。通过手动修改 h1 的块首指针，模拟了 DWORD SHOOT 漏洞，展示了堆溢出攻击的原理。该实验突出了堆管理中的潜在安全风险，强调了在编程中需要谨慎处理堆内存的分配和释放。



```
#include <windows.h>
main()
{
    HLOCAL h1, h2, h3, h4, h5, h6;
    HANDLE hp;
    hp = HeapCreate(0, 0x1000, 0x10000); // 创建自主管理的堆
    h1 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 8); // 从堆里申请空间
    h2 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 8);
    h3 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 8);
    h4 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 8);
    h5 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 8);
    h6 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 8);

    _asm int 3 // 手动增加 int3 中断指令，会让调试器在此处中断
    // 依次释放奇数堆块，避免堆块合并
    HeapFree(hp, 0, h1); // 释放堆块
    HeapFree(hp, 0, h3);
    HeapFree(hp, 0, h5); // 现在 freelist[2] 有 3 个元素

    h1 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 8);

    return 0;
}
```

• 代码流程解析：

<1>运行实验代码，程序在 int3 断点处停止。此时，已通过 HeapCreate 创建了一个大小为 0x1000 的堆区。使用 HeapAlloc 分配了 6 个堆块（h1 至 h6），每个堆块申请 8 字节用户空间，加上 8 字节的块首信息，实际每个堆块占用 16 字节。初始化完成后，堆区中连续排列了 6 个 16 字节的堆块。

<2>程序依次释放了奇数次申请的堆块（h1、h3、h5），以防止堆块合并。释放后，这三个 16 字节的空闲堆块被放入空闲链表（freelist）中。由于它们的大小为 16 字节，因此被链接到 freelist[2] 所标识的空表中。

<3>空闲链表是一个双向链表，每个空闲堆块的块首中包含前向指针（Flink）和后向指针（Blink），用于链接前后空闲堆块。

释放后，freelist[2] 的结构：freelist[2] -> h1 -> h3 -> h5。h1 的 Flink 指向 h3，Blink 指向 freelist[2]；h3 的 Flink 指向 h5，Blink 指向 h1；h5 的 Flink 指向

freelist[2], Blink 指向 h3。

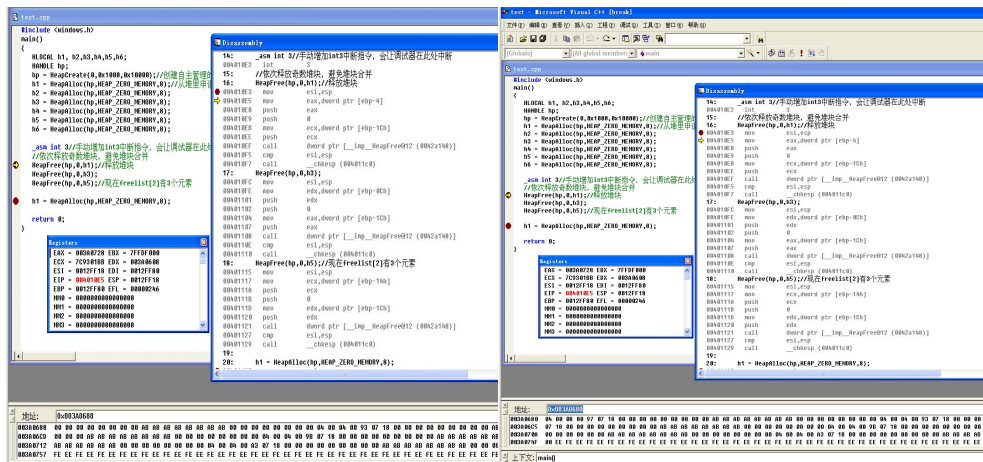
<4>程序再次申请 8 字节的堆块（实际分配 16 字节）。系统会从 freelist[2]中取出第一个空闲堆块(h1)进行分配。分配后，freelist[2]的链表结构更新为：freelist[2] -> h3 -> h5

<5>在分配 h1 之前,如果手动修改 h1 的块首中的前向指针(Flink)和后向指针(Blink),则可以触发 DWORD SHOOT。

• 通过调试器观察堆内存的动态变化：

在执行 HeapFree (hp, 0, h1) 语句前：

堆句柄 hp 的值为 0x003A0000，表示堆区的基地址。堆块 h1 的地址为 0x003A0688。堆块 h1 的块首地址为 0x003A0680（即 h1 地址减去 8 字节的块首大小）。

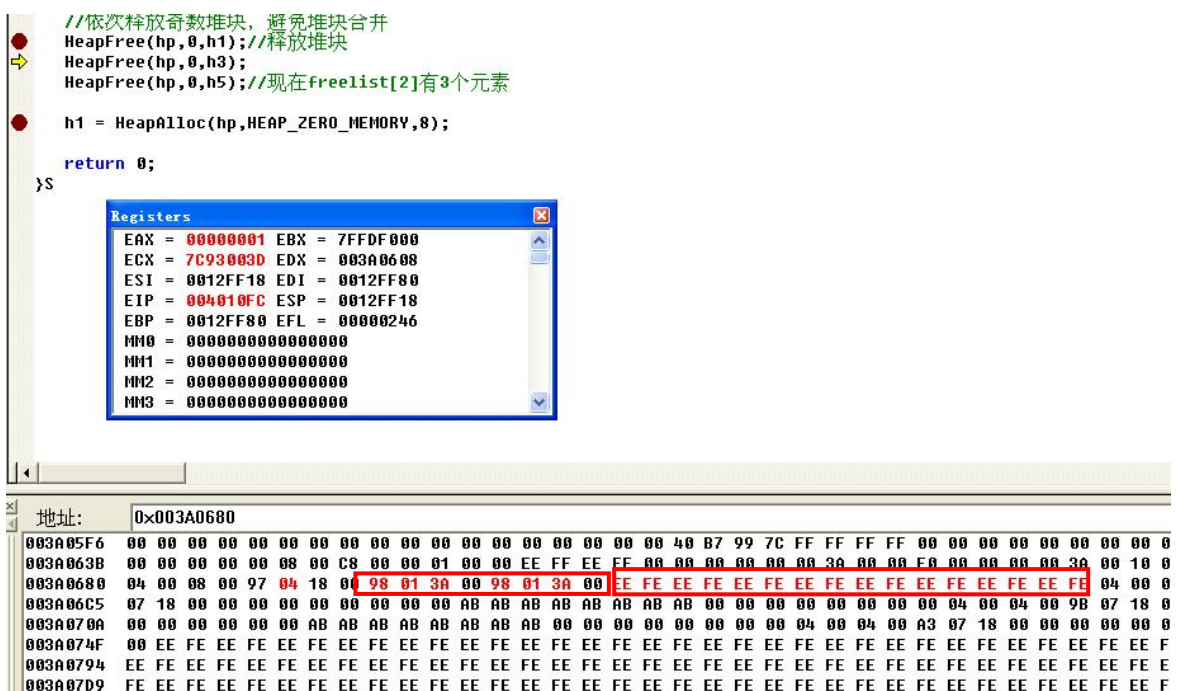


执行 HeapFree (hp, 0, h1) 语句后的变化：

h1 被释放，其块首地址为 0x003A0688。freelist[2]更新，地址为 0x003A0198。Flink 和 Blink 均指向 h1 的块首地址 0x003A0688。h1 的块首信息位于 0x003A0680。

空闲链表结构：

freelist[2] (0x003A0198) -> h1 (0x003A0688) -> freelist[2] (0x003A0198)



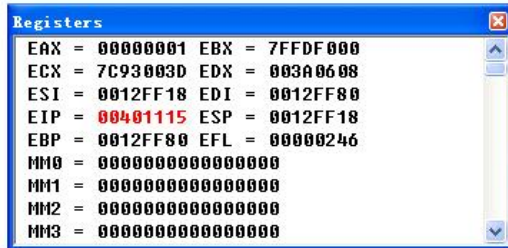

```

//依次释放奇数堆块，避免堆块合并
HeapFree(hp,0,h1);//释放堆块
HeapFree(hp,0,h3);
HeapFree(hp,0,h5);//现在freelist[2]有3个元素

h1 = HeapAlloc(hp,HEAP_ZERO_MEMORY,8);

return 0;
}S

```



地址:	0x003A0680
003A05B1	00 00 00 00 00 00 00 00 C8 05 3A 00 00 00 00 00 00 00 00 00 00 00 00 D8 05 3A 00 00 00 00
003A05F6	00 40 B7 99 7C FF FF FF FF 00 00 00
003A063B	00 00 00 00 00 00 00 00 C8 00 00 01 00 00 EE FF EE FF 00 00 00 00 00 00 00 3A 00 00 F0 00 00
003A0680	04 00 08 00 97 04 18 00 C8 06 3A 00 08 01 3A 00 EE FE EE FE EE FE EE FE EE FE EE FE EE FE EE
003A06C5	04 18 00 98 01 3A 00 88 06 3A 00 EE FE EE FE EE FE EE FE EE FE EE FE EE FE EE FE EE FE EE 04 00
003A070A	00 00 00 00 00 00 00 AB AB AB AB AB AB AB 00 00 00 00 00 00 00 00 00 00 04 00 04 00 A3 07 18

返回 0x003A0680 查看 h1 的变化:

h1 的前向指针 (Flink) 从原来的 freelist[2] 地址 0x003A0198 更新为 0x003A06C8, 即 h3 的块身首地址。h1 的后向指针 (Blink) 仍指向 freelist[2] 的地址 0x003A0198。

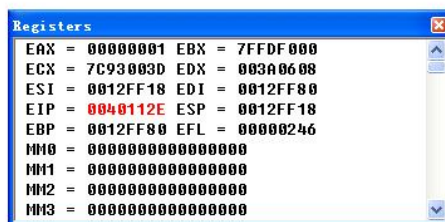
```

//依次释放奇数堆块，避免堆块合并
HeapFree(hp,0,h1);//释放堆块
HeapFree(hp,0,h3);
HeapFree(hp,0,h5);//现在freelist[2]有3个元素

h1 = HeapAlloc(hp,HEAP_ZERO_MEMORY,8);

return 0;
}S

```



地址:	00x003A0198
003A00C9	00 00
003A010E	00 00
003A0153	00 00 00 00 00 00 10 00
003A0198	88 06 3A 00 08 07 3A 00 A0 01 3A 00 A0 01 3A 00 A8 01 3A 00 A8 01 3A 00 B0 01 3A 00 B0 01
003A01DD	01 3A 00 E0 01 3A 00 E0 01 3A 00 E8 01 3A 00 E8 01 3A 00 F0 01 3A 00 F0 01 3A 00 F8 01 3A
003A0222	3A 00 20 02 3A 00 28 02 3A 00 28 02 3A 00 30 02 3A 00 30 02 3A 00 38 02 3A 00 38 02 3A 00
003A0267	00 68 02 3A 00 68 02 3A 00 70 02 3A 00 70 02 3A 00 78 02 3A 00 78 02 3A 00 80 02 3A 00 80
003A02AC	A8 02 3A 00 B0 02 3A 00 B0 02 3A 00 B8 02 3A 00 B8 02 3A 00 C0 02 3A 00 C0 02 3A 00 C8 02
003A02F4	00 00

执行 HeapFree(hp, 0, h5) 语句前后的变化: 执行前, h5 的块身首地址为 0x003A0708。

空闲链表中已有 h1 和 h3, 结构为:

freelist[2] (0x003A0198) -> h1 (0x003A0688) -> h3 (0x003A06C8) -> freelist[2]

执行后，h5 被释放，其块身首地址为 0x003A0708。h5 被链接到空闲链表的末尾，h3 的 Flink 更新为指向 h5。h5 的 Flink 指向 freelist[2]，Blink 指向 h3。freelist[2] 的 Blink 更新为指向 h5。

继续按 F10 执行后的变化：0x003A06C8 是 h3 的块身首地址。h3 是 freelist[2] 中的第一个空闲块。

```
//从堆中释放堆块，避免堆块内存泄漏
HeapFree(hp, 0, h1); //释放堆块
HeapFree(hp, 0, h3);
HeapFree(hp, 0, h5); //现在freelist[2]有3个元素

h1 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 8);

return 0;
}
```

输入 h3 的地址 0x003A06C8 后:

h3 的后向指针 (Blink) 更新为 0x003A0198, 即 freelist[2] 的地址。空闲链表结构为, h3 的 Flink 仍指向 h5 的块身首地址 0x003A0708。h3 的 Blink 指向 freelist[2] 的地址 0x003A0198。

执行 HeapAlloc (hp, HEAP_ZERO_MEMORY, 8) 后的变化:

从 freelist[2] 中摘下 h1, 分配 8 字节 (实际 16 字节) 的堆块, 从 freelist[2] 中取出第一个空闲块 h1。h1 的块身首地址为 0x003A0688。

空闲链表结构更新:

freelist[2] (0x003A0198) -> h3 (0x003A06C8) -> h5 (0x003A0708) -> freelist[2] (0x003A0198)

h1 被分配并从空闲链表中移除。freelist[2] 的 Flink 更新为指向 h3, h3 的 Blink 更新为指向 freelist[2]。空闲链表中剩余的空闲堆块为 h3 和 h5, 按顺序链接。

• DWORD SHOOT 攻击:

通过修改 h1 的前向指针 (Flink) 和后向指针 (Blink), 使其指向目标地址和特定数值。当 HeapAlloc 从 freelist[2] 中摘取 h1 时, 触发 DWORD SHOOT, 将目标地址的内容修改为特定数值。攻击效果是实现任意内存写入, 可能用于覆盖关键数据 (如函数指针、返回地址等), 从而执行任意代码或提权。

心得体会:

1. 掌握了堆块的分配与释放过程, 以及堆管理器如何通过块首信息 (如前向指针 Flink 和后向指针 Blink) 维护堆的状态。实验验证了堆块在分配和释放时, 操作系统会动态更新空闲链表 (freelist) 以反映堆的当前状态。
2. 了解了空闲链表的结构与存储方式, 空闲链表采用双向链表结构, 确保了内存的高效利用和回收。通过观察 freelist[2] 的变化, 理解了不同大小的空闲堆块如何被组织和管理。
3. 实验模拟了通过堆溢出漏洞篡改空闲堆块的指针, 触发 DWORD SHOOT 攻击的过程。理解了攻击者如何利用堆管理器的双向链表操作实现任意内存写入, 从而可能执行恶意代码或提权。
4. 本次实验不仅加深了对堆管理机制的理解, 还揭示了堆溢出漏洞的实际危害, 为后续的安全编程和漏洞防护提供了重要的实践基础。